

SAHAYOG SARTHI

On-Demand Worker Booking Platform

Software Requirements Specification (SRS) & Complete Developer Guide

Document Version: 1.0

Classification: Internal Development Reference

Platform: Web + Mobile (Android & iOS)

1. Executive Summary

Sahayog Sarthi is a location-based, on-demand service marketplace that connects customers with nearby skilled workers such as electricians, plumbers, carpenters, painters, AC technicians, mechanics, cleaners, drivers, and other professionals.

The platform enables real-time worker discovery based on GPS proximity, instant booking, live tracking, in-app communication, and seamless digital payments — all within a single unified ecosystem.

Sahayog Sarthi is structured around three distinct user panels:

- Admin Panel – Platform governance, verification, analytics, and operations management.
- Customer Panel – Worker discovery, booking, tracking, payment, and review.
- Worker Panel – Job acceptance, KYC onboarding, earnings management, and live availability.

This document serves as the complete Software Requirements Specification (SRS) and developer guide for building the Sahayog Sarthi platform from the ground up.

2. Project Objectives

1. Connect customers with verified, nearby skilled workers in real time.
2. Provide GPS-based worker availability and intelligent discovery.
3. Enable smooth end-to-end booking, tracking, and payment flows.
4. Generate employment opportunities for skilled professionals.
5. Ensure transparency, safety, and trust via worker KYC verification.
6. Build a scalable, extensible platform ready for startup-grade deployment.

3. Scope of the System

3.1 In Scope

- Customer registration, login, and profile management.
- Worker registration with KYC document verification.
- GPS-based nearby worker search and filtering.
- Real-time booking workflow with acceptance/rejection.
- Live location tracking of workers during active bookings.
- In-app chat between customer and worker.
- Payment gateway integration (Razorpay — UPI, Cards, Net Banking).
- Admin dashboard with analytics, reporting, and user management.
- Rating and review system for completed jobs.
- Push notification system for bookings and updates.

3.2 Out of Scope (Future Enhancements)

- Video calling with workers.
- AI-powered cost estimation engine.
- Franchise management module.
- Voice assistant booking.
- Worker insurance management.
- Multi-language (i18n) support.

4. User Roles and Responsibilities

4.1 Administrator

The Administrator is the platform owner who governs the entire ecosystem. Key responsibilities include:

- Manage and verify worker registration and KYC documents.
- Monitor all active, completed, and cancelled bookings.
- Handle customer and worker complaints and disputes.
- View revenue reports, platform analytics, and heatmaps.
- Manage service categories and platform configurations.
- Block or unblock user accounts as needed.
- Send broadcast notifications to users or workers.

4.2 Customer

Customers are end-users who hire workers for various service needs. Key capabilities include:

- Register/login via mobile OTP.
- Detect current location using GPS.
- Search and filter nearby workers by distance, rating, price, or category.
- Book a worker and track their live location.
- Communicate with the worker via in-app chat.
- Pay digitally and receive booking receipts.
- Submit ratings and written reviews post-service.
- Access complete booking history.

4.3 Worker

Workers are skilled professionals who offer services through the platform. Key capabilities include:

- Register with personal, professional, and KYC details.
- Toggle availability status (Online/Offline).
- Receive booking requests and Accept or Reject them.
- Share live GPS location during active jobs.
- View earnings dashboard and withdraw to bank/wallet.
- View job history and customer feedback.

5. Functional Requirements

5.1 Authentication Module

- FR-01: The system shall allow customers to register using mobile number + OTP.
- FR-02: The system shall allow workers to register with name, mobile, service category, and document upload.
- FR-03: JWT-based session management shall be implemented for all panels.
- FR-04: Admin login shall be username + password with hashed credential storage.

5.2 Location & Worker Discovery Module

- FR-05: The system shall detect the customer's current GPS coordinates on app launch.
- FR-06: The system shall query and display workers within a configurable radius (default: 10 km).
- FR-07: Workers shall be displayed with name, category, distance (km), estimated arrival time, and rating.
- FR-08: The customer shall be able to filter results by distance, rating, price range, and category.

5.3 Booking Module

- FR-09: The customer shall be able to select a worker and confirm a booking.
- FR-10: The worker shall receive an instant push notification with booking details.
- FR-11: The worker shall be able to Accept or Reject the booking within 60 seconds.
- FR-12: On acceptance, the booking status shall update to 'In Progress'.
- FR-13: The system shall allow customers to cancel bookings before acceptance.

5.4 Live Tracking Module

- FR-14: Upon booking acceptance, real-time worker location shall be shared with the customer.
- FR-15: Worker location shall update every 5 seconds via Socket.IO.
- FR-16: The customer app shall display the worker's route on Google Maps.

5.5 Payment Module

- FR-17: Razorpay shall be integrated for UPI, Debit/Credit Card, and Net Banking.
- FR-18: Payment shall be triggered upon service completion.
- FR-19: Worker earnings shall be credited to their platform wallet after platform commission deduction.
- FR-20: Workers shall be able to withdraw wallet balance to their registered bank account.

5.6 Communication Module

- FR-21: In-app chat shall be available between customer and worker for an active booking.
- FR-22: Chat messages shall be stored and linked to the booking record.

5.7 Rating & Review Module

- FR-23: After service completion, the customer shall be prompted to rate the worker (1–5 stars).
- FR-24: Optional written review shall be allowed alongside the star rating.
- FR-25: Worker's average rating shall be recalculated and updated in real time.

5.8 Admin Module

- FR-26: Admin shall be able to approve or reject worker KYC submissions.
- FR-27: Admin shall view a live dashboard with platform metrics (active bookings, revenue, new registrations).
- FR-28: Admin shall be able to block/unblock customer or worker accounts.
- FR-29: Admin shall manage complaint tickets raised by users.

6. Non-Functional Requirements

Category	Requirement	Target
Performance	API response time	< 300 ms (95th percentile)
Scalability	Concurrent users supported	10,000+ simultaneous users
Availability	Platform uptime	99.9% SLA
Security	Password & data encryption	bcrypt + TLS 1.3
GPS Accuracy	Location precision	± 10 metres
Compatibility	Browser and device support	Chrome, Safari, Firefox; Android 8+, iOS 13+
Data Retention	Booking and payment records	Minimum 5 years

7. System Architecture

7.1 Architecture Pattern

Sahayog Sarthi follows a three-tier, microservices-ready architecture:

- Presentation Layer: Next.js (React) web app + React Native mobile apps.
- Application Layer: Node.js + Express.js REST API server with Socket.IO for real-time events.
- Data Layer: MongoDB (primary database) + Redis (caching + session store).

7.2 High-Level Component Diagram

The system comprises the following primary components:

Component	Description
API Gateway	Single entry point; handles routing, rate limiting, and auth middleware.
Auth Service	Manages OTP generation, JWT issuance, and session validation.
Location Service	Processes GPS data and performs geospatial queries using MongoDB 2dsphere indexes.
Booking Service	Orchestrates the booking lifecycle: creation, acceptance, tracking, and completion.
Payment Service	Integrates with Razorpay; manages wallet credits, debits, and withdrawals.
Notification Service	Sends push notifications via Firebase Cloud Messaging (FCM).
Chat Service	Real-time messaging over Socket.IO with message persistence in MongoDB.
Admin Service	Dashboard APIs for analytics, KYC review, and complaint management.

7.3 Communication Flow

Client (Browser/Mobile App)
↓ HTTPS REST / WebSocket
API Gateway (Express.js + Socket.IO)
↓ Internal Service Calls
Microservices (Auth, Booking, Location, Payment, Chat, Notification)
↓ Mongoose ODM
MongoDB Atlas (Primary Store) + Redis (Cache)
↓ External APIs
Google Maps API | Razorpay | Firebase FCM

8. Database Design

The platform uses MongoDB (NoSQL) as its primary data store. The following collections form the core data model.

8.1 Users Collection

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier.
name	String	Full name of the customer.
mobile	String	10-digit mobile number (unique, indexed).
email	String	Email address (optional, unique if provided).
profileImage	String	URL of uploaded profile image.
location	GeoJSON	Last known GPS coordinates {type, coordinates}.
address	String	Saved default address.
isBlocked	Boolean	Account block status (default: false).
createdAt	Date	Account creation timestamp.

8.2 Workers Collection

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier.
name	String	Full name of the worker.
mobile	String	Mobile number (unique, indexed).
serviceCategory	String	Service type (e.g., Electrician, Plumber).
experience	Number	Years of professional experience.
rating	Number	Average rating (0.0 – 5.0), computed field.
totalRatings	Number	Count of ratings received.
location	GeoJSON	Live GPS location {type: 'Point', coordinates: [lng, lat]}.
isAvailable	Boolean	Online/Offline toggle status.
kycStatus	Enum	pending approved rejected
documents	Array	Array of KYC document URLs.
walletBalance	Number	Current wallet balance in INR.
hourlyRate	Number	Worker's charge per hour in INR.
isBlocked	Boolean	Account block status.

8.3 Bookings Collection

Field	Type	Description
_id	ObjectId	Unique booking identifier.
customerId	ObjectId	Reference to Users collection.
workerId	ObjectId	Reference to Workers collection.
serviceType	String	Service category for this booking.
status	Enum	pending accepted in_progress completed cancelled
scheduledAt	Date	Requested date and time for service.
completedAt	Date	Actual service completion timestamp.
amount	Number	Total billed amount in INR.
paymentStatus	Enum	pending paid refunded
customerAddress	String	Service address provided by customer.
notes	String	Optional notes or instructions from customer.

8.4 Payments Collection

Field	Type	Description
_id	ObjectId	Payment record identifier.
bookingId	ObjectId	Reference to Bookings collection.
razorpayOrderId	String	Order ID returned by Razorpay.
razorpayPaymentId	String	Payment ID after successful transaction.
amount	Number	Transaction amount in INR (paise in Razorpay).
status	Enum	created paid failed refunded
method	String	upi card netbanking wallet
createdAt	Date	Payment initiation timestamp.

8.5 Reviews Collection

Field	Type	Description
bookingId	ObjectId	One review per booking (unique constraint).
customerId	ObjectId	Reviewer reference.
workerId	ObjectId	Worker being reviewed.
rating	Number	Star rating (1 to 5).
comment	String	Optional written feedback.
createdAt	Date	Review submission timestamp.

9. API Specification

All API endpoints use REST conventions. Base URL: <https://api.sahayogsarthi.com/v1>

9.1 Authentication APIs

Method	Endpoint	Description
POST	/auth/send-otp	Send OTP to customer's mobile number.
POST	/auth/verify-otp	Verify OTP and return JWT token.
POST	/auth/register/worker	Worker registration with details and documents.
POST	/auth/admin/login	Admin login with username and password.
POST	/auth/logout	Invalidate active session/token.

9.2 Worker APIs

Method	Endpoint	Description
GET	/workers/nearby	Fetch workers near given coordinates. Params: lat, lng, radius, category.
GET	/workers/:id	Get full profile of a specific worker.
PUT	/workers/:id/availability	Toggle worker online/offline status.
PUT	/workers/:id/location	Update worker's live GPS coordinates.
GET	/workers/:id/earnings	Fetch earnings summary for the worker.

9.3 Booking APIs

Method	Endpoint	Description
POST	/bookings	Create a new booking request.
GET	/bookings/:id	Fetch booking details by ID.
PUT	/bookings/:id/accept	Worker accepts the booking.
PUT	/bookings/:id/reject	Worker rejects the booking.
PUT	/bookings/:id/complete	Mark booking as completed.
PUT	/bookings/:id/cancel	Customer cancels a pending booking.
GET	/bookings/history	Get paginated booking history for logged-in user.

9.4 Payment APIs

Method	Endpoint	Description
POST	/payments/create-order	Create Razorpay order for a booking.
POST	/payments/verify	Verify Razorpay payment signature.
POST	/payments/withdraw	Worker requests wallet withdrawal.
GET	/payments/transactions	List transaction history for user.

10. Technology Stack

Layer	Technology	Purpose
Frontend	Next.js + React.js	Customer and Admin web UI (SSR/SSG).
Styling	Tailwind CSS	Utility-first, responsive UI styling.
Mobile	React Native	Cross-platform Android and iOS apps.
Backend	Node.js + Express.js	REST API server and business logic.
Real-Time	Socket.IO	Live tracking, chat, booking notifications.
Database	MongoDB Atlas	Primary NoSQL data store with geospatial support.
Cache	Redis	Session storage, OTP TTL, rate limiting.
Auth	JWT + OTP (SMS)	Stateless authentication and mobile verification.
Maps	Google Maps API	Worker map display, live tracking, routing.
Payments	Razorpay	UPI, Cards, Net Banking payment processing.
Notifications	Firebase FCM	Push notifications for bookings and updates.
File Storage	AWS S3 / Cloudinary	Worker KYC documents and profile images.
SMS Gateway	Twilio / MSG91	OTP delivery to mobile numbers.
Deployment	Vercel + AWS EC2	Frontend on Vercel; backend on AWS EC2.

11. Development Roadmap

Phase 1 — Requirement Analysis (Week 1)

- Finalize all user stories and acceptance criteria.
- Define service categories and platform commission structure.
- Confirm external API integrations (Google Maps, Razorpay, FCM).
- Create wireframes and information architecture.

Phase 2 — System & Database Design (Week 2)

- Design MongoDB collections and define all indexes (including 2dsphere for GPS).
- Define API contracts (request/response schema for each endpoint).
- Design Socket.IO event schema for real-time features.
- Plan Redis caching strategy for OTP and session data.

Phase 3 — UI/UX Design (Week 3)

- Design high-fidelity screens for Customer App (12 screens).
- Design Worker App screens (8 screens).
- Design Admin Panel dashboard and management screens.
- Define design system: color palette, typography, component library.

Phase 4 — Frontend Development (Weeks 4–6)

- Set up Next.js project with Tailwind CSS and folder structure.
- Build Authentication screens (OTP login, registration).
- Implement Home Page with category grid and search.
- Build Nearby Workers listing with Google Maps integration.
- Develop Worker Detail Page and Booking Confirmation screen.
- Implement Payment flow with Razorpay SDK.
- Build Customer Profile, Booking History, and Reviews screens.

Phase 5 — Backend Development (Weeks 6–8)

- Initialize Node.js + Express.js project with ESM modules.
- Set up MongoDB Atlas connection and define Mongoose schemas.
- Implement Auth Service (OTP via MSG91, JWT issuance).
- Build Location Service with \$geoNear aggregation pipeline.
- Develop Booking Service with full lifecycle state machine.
- Integrate Razorpay for order creation and webhook verification.
- Build Admin APIs for KYC, user management, and analytics.

Phase 6 — Real-Time Features (Weeks 8–10)

- Integrate Socket.IO server with Express.js.
- Implement booking request/acceptance flow over WebSocket.
- Build live worker location tracking (5-second update interval).
- Implement real-time in-app chat with message persistence.
- Integrate Firebase FCM for push notifications on booking events.

Phase 7 — Worker Recommendation Engine (Weeks 10–11)

- Implement scoring algorithm: $\text{Score} = (0.4 \times \text{Rating}) + (0.3 \times \text{Experience}) + (0.3 \times \text{DistanceScore})$.
- Apply score-based sorting on the nearby workers query.
- Add smart search by worker name, category, and availability.

Phase 8 — Admin Panel Development (Week 11)

- Build Admin dashboard with live metrics (bookings, revenue, registrations).
- Implement KYC document review and approval/rejection workflow.
- Build complaint management with status tracking.
- Develop revenue reports with date-range filtering and CSV export.

Phase 9 — Testing (Weeks 12–13)

- Unit testing: Jest for all service modules.
- Integration testing: Supertest for all API endpoints.
- Load testing: k6 for 1,000+ concurrent users.
- Security testing: JWT expiry, CORS policy, input sanitization, SQL/NoSQL injection prevention.
- GPS accuracy testing: Location precision validation across devices.
- Payment testing: Razorpay test mode end-to-end transaction flows.

Phase 10 — Deployment (Week 14)

- Deploy Next.js frontend to Vercel with environment variables.
- Deploy Node.js backend to AWS EC2 with PM2 process manager.
- Configure MongoDB Atlas production cluster with IP whitelisting.
- Set up Redis on AWS ElastiCache or Railway Redis.
- Configure Nginx as reverse proxy with SSL termination (Let's Encrypt).
- Set up CI/CD pipeline (GitHub Actions) for automated deployment.

12. Step-by-Step Build Guide

Step 1 — Project Setup

Initialize the monorepo structure:

- sahayog-sarthi/
 - frontend/ (Next.js customer + admin)
 - backend/ (Node.js API server)
 - mobile/ (React Native app)
 - docs/ (API documentation)

Backend initialization commands:

- mkdir backend && cd backend
- npm init -y
- npm install express mongoose socket.io jsonwebtoken bcryptjs razorpay firebase-admin multer redis dotenv cors
- npm install -D nodemon jest supertest

Step 2 — Database Connection & Schemas

Create /backend/src/config/db.js to connect MongoDB Atlas using MONGO_URI from .env. Define Mongoose schemas for all collections described in Section 8. Critically, add a 2dsphere index to the Workers collection location field to enable geospatial queries:

- workerSchema.index({ location: '2dsphere' });

Step 3 — Authentication Implementation

Create /backend/src/services/authService.js:

7. Generate 6-digit OTP and store in Redis with 5-minute TTL keyed by mobile number.
8. Send OTP via MSG91 or Twilio SMS API.
9. On OTP verification, find or create user in MongoDB and return signed JWT (expires in 7 days).
10. Implement authMiddleware.js to verify JWT on protected routes.

Step 4 — Geospatial Worker Search

In /backend/src/services/locationService.js, implement the nearby worker query using MongoDB's \$geoNear aggregation stage. This is the core of the worker discovery feature:

- Accept latitude, longitude, radius (metres), and optional category filter.
- Use db.workers.aggregate([{\$geoNear: { near: { type: 'Point', coordinates: [lng, lat] }, distanceField: 'distance', maxDistance: radius, query: { isAvailable: true, kycStatus: 'approved' }, spherical: true } }])
- Return sorted results with computed distance in km and estimated arrival time.

Step 5 — Booking State Machine

The booking lifecycle is managed through the following state transitions:

- pending → accepted (worker accepts within 60 seconds)
- pending → cancelled (customer cancels or worker timeout)
- accepted → in_progress (worker arrives and starts job)
- in_progress → completed (worker marks job done)
- completed → paid (payment confirmed by Razorpay webhook)

Step 6 — Socket.IO Real-Time Events

Define the following Socket.IO event schema:

Event Name	Direction	Payload
booking:new	Server → Worker	{ bookingId, customerName, serviceType, address }
booking:accepted	Server → Customer	{ bookingId, workerName, workerLocation }
location:update	Worker → Server	{ bookingId, lat, lng }
location:broadcast	Server → Customer	{ lat, lng, eta }
chat:message	Bidirectional	{ bookingId, senderId, message, timestamp }
booking:completed	Server → Customer	{ bookingId, amount }

Step 7 — Razorpay Payment Integration

11. Customer triggers payment → Backend calls `razorpay.orders.create({ amount, currency: 'INR' })`.
12. Frontend receives `order_id` and opens Razorpay checkout modal.
13. Customer completes payment; Razorpay sends `razorpay_payment_id` and `razorpay_signature`.
14. Backend verifies signature using `crypto.createHmac('sha256', secret).update(orderId + '|' + paymentId).digest('hex')`.
15. On signature match, mark booking payment as 'paid' and credit worker wallet (minus platform commission).

Step 8 — Frontend Next.js Structure

Recommended page structure for the Next.js frontend:

- `/pages/index.js` — Landing page with categories and search.
- `/pages/workers/index.js` — Nearby workers listing with map.
- `/pages/workers/[id].js` — Worker detail and booking form.

- /pages/bookings/[id].js — Live tracking page.
- /pages/payment/[bookingId].js — Payment page.
- /pages/profile.js — Customer profile and history.
- /pages/admin/ — Admin panel with protected routes.

Step 9 — Google Maps Integration

16. Install: `npm install @react-google-maps/api`
17. Wrap app with `<LoadScript googleMapsApiKey={process.env.NEXT_PUBLIC_MAPS_KEY}>`.
18. On nearby workers page, render `<GoogleMap>` centered on customer's location.
19. Place `<Marker>` for each nearby worker using their GPS coordinates.
20. On active booking, subscribe to `location:broadcast` socket event and update marker position in real time.
21. Use `DirectionsService` to display estimated route from worker to customer.

13. Security Guidelines

- All API endpoints must be served over HTTPS (TLS 1.3 minimum).
- Passwords must be hashed using bcrypt with a minimum cost factor of 12.
- JWT tokens must have a 7-day expiry; refresh tokens must be stored in HttpOnly cookies.
- OTPs must expire in 5 minutes and be invalidated after first successful use.
- Input validation must be enforced using Joi or express-validator on all POST/PUT bodies.
- MongoDB queries must use parameterized queries via Mongoose to prevent injection.
- File uploads must be validated for type (PDF/JPG/PNG only) and size (max 5 MB) before S3 upload.
- API rate limiting must be implemented using express-rate-limit (100 requests/minute per IP).
- CORS must restrict origins to known frontend domains only.
- Admin panel must enforce two-factor authentication (2FA) for login.

14. Environment Variables Reference

Variable	Description
MONGO_URI	MongoDB Atlas connection string.
JWT_SECRET	Secret key for JWT signing (minimum 32 characters).
JWT_EXPIRES_IN	JWT expiry duration (e.g., 7d).
REDIS_URL	Redis connection string for OTP and session cache.
RAZORPAY_KEY_ID	Razorpay API Key ID from dashboard.
RAZORPAY_KEY_SECRET	Razorpay API Key Secret for signature verification.
GOOGLE_MAPS_API_KEY	Google Maps API key (backend geocoding).
NEXT_PUBLIC_MAPS_KEY	Google Maps API key for frontend (public, Maps JS API).
FCM_SERVER_KEY	Firebase Cloud Messaging server key.
MSG91_AUTH_KEY	MSG91 API key for OTP SMS delivery.
AWS_S3_BUCKET	S3 bucket name for document storage.
AWS_ACCESS_KEY_ID	AWS IAM access key for S3 operations.
AWS_SECRET_ACCESS_KEY	AWS IAM secret key.
PLATFORM_COMMISSION	Platform commission percentage (e.g., 0.15 for 15%).

15. Deployment Guide

15.1 Frontend Deployment (Vercel)

22. Push Next.js project to a GitHub repository.
23. Sign in to vercel.com and click 'New Project'.
24. Import the GitHub repository and set the root directory to /frontend.
25. Add all NEXT_PUBLIC_* environment variables in the Vercel dashboard.
26. Vercel automatically builds and deploys on every push to the main branch.

15.2 Backend Deployment (AWS EC2)

27. Launch an EC2 instance (Ubuntu 22.04 LTS, t3.medium minimum).
28. SSH in and install Node.js 20.x, PM2, and Nginx.
29. Clone the backend repository, install dependencies, and create the .env file.
30. Start the server with: `pm2 start src/index.js --name sahayog-api`
31. Configure Nginx as a reverse proxy to forward port 80/443 to the Node.js port (default 5000).
32. Obtain SSL certificate using: `sudo certbot --nginx -d api.sahayogsarthi.com`

15.3 Database (MongoDB Atlas)

33. Create a free or dedicated cluster on cloud.mongodb.com.
34. Create a database user with read/write permissions.
35. Whitelist the EC2 instance's static IP address.
36. Copy the connection string and set it as MONGO_URI in the EC2 .env file.
37. Enable Atlas backup for automatic daily snapshots.

16. Complete Booking Sequence

The following outlines the complete step-by-step flow of a booking from discovery to review:

38. Customer opens the app; GPS coordinates are captured and sent to the Location Service.
39. Location Service queries MongoDB using \$geoNear and returns a list of available, KYC-approved workers sorted by recommendation score.
40. Customer views the worker list on the map, selects a worker, and taps 'Book Now'.
41. Frontend calls POST /bookings with workerId, serviceType, and address payload.
42. Booking Service creates a booking record (status: pending) and emits booking:new over Socket.IO to the worker's device.
43. Worker receives a push notification and in-app booking request popup. The worker must respond within 60 seconds.
44. Worker taps 'Accept' → PUT /bookings/:id/accept is called → status transitions to accepted.
45. Socket.IO emits booking:accepted to the customer app; live tracking view opens.
46. Worker's device begins sending location:update events every 5 seconds; customer sees worker moving on map.
47. Worker arrives at the service location and begins work. Worker taps 'Start Job' → status: in_progress.
48. On job completion, worker taps 'Complete Job' → PUT /bookings/:id/complete → status: completed.
49. Customer receives a payment prompt; Razorpay checkout modal opens.
50. Customer completes payment; POST /payments/verify confirms signature and marks status: paid.
51. Worker wallet is credited (amount minus platform commission).
52. Customer is prompted to rate and review the worker. Review is saved and worker's average rating is recalculated.

17. Planned Future Enhancements

#	Feature	Description
1	AI Worker Matching	Machine learning model trained on booking history to predict best-fit workers.
2	Video Call Before Booking	Customers can video consult a worker before confirming the job.
3	Multi-language Support	i18n support for Hindi, Tamil, Telugu, and other regional languages.
4	Subscription Plans	Monthly/annual plans for customers offering discounted service rates.
5	Worker Insurance	On-job accident insurance integrated with insurance provider APIs.
6	Voice Assistant Booking	Book services using voice commands via Google Assistant or Alexa.
7	AI Cost Estimator	Automatically suggest job cost estimates based on service type and location.
8	SOS Emergency Booking	One-tap emergency booking that bypasses normal flow for urgent situations.
9	Franchise Management	City-level franchise portal for independent operators.
10	Worker Attendance Tracking	Daily availability and shift management for workers.

18. Glossary

Term	Definition
SRS	Software Requirements Specification — formal document describing system behaviour.
GPS	Global Positioning System — used to detect real-time device location.
KYC	Know Your Customer — identity verification process for workers.
JWT	JSON Web Token — stateless authentication mechanism.
OTP	One-Time Password — temporary code sent via SMS for authentication.
REST	Representational State Transfer — architectural style for APIs.
FCM	Firebase Cloud Messaging — push notification service by Google.
2dsphere	MongoDB index type for geospatial (spherical) queries.
Socket.IO	Library for real-time, bidirectional communication over WebSocket.
SSR	Server-Side Rendering — web page rendered on the server for performance and SEO.